Advanced Procedures

Computer Organization and Assembly Language

Computer Science Department

National University of Computer and Emerging Sciences Islamabad

Outlines

- Introduction
- Stack Frames
- Recursion

INTRODUCTION

- In MASM, subroutines are called ***procedures***.
- Values passed to a subroutine by a calling program are called ***arguments***.
- When the values are received by the called subroutine, they are called ***parameters***.
- Parameters can be passed both in registers and on the stack.

STACK APPLICATION

- - Runtime memory reqs:
- Arguments/Parameters
- Return Address
- EBP
- Local Variables
- Registers Storage

8.2 Stack Frames

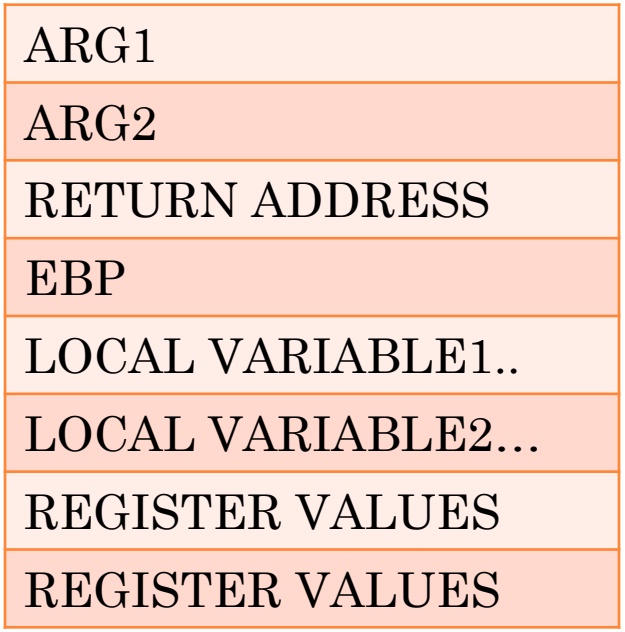
- A *stack frame* (or *activation record*) is the area of the stack set aside for passed arguments, subroutine return address, local variables, and saved registers.

- The Stack Frame is created by the following steps:

1. Passed arguments are pushed on the stack.
2. The subroutine is called, causing the subroutine return address to be pushed on the stack.
3. As the subroutine begins to execute, EBP is pushed on the stack..

4. EBP is set equal to ESP.
   - From this point on, EBP acts as a base reference for all of the subroutine parameters
5. If there are local variables, ESP is decremented to reserve space for the variables on the stack.
6. If any registers need to be saved, they are pushed on the stack.

- The structure of a stack frame is directly affected by a program's memory model and its choice of argument passing convention.


ARG1
ARG2
RETURN ADDRESS
EBP
LOCAL VARIABLE1..
LOCAL VARIABLE2…
REGISTER VALUES
REGISTER VALUES

Disadvantages of register parameters

- In earlier chapters, our subroutines received register parameters.
- Because registers' use is multipurpose, the registers used as parameters must
  - first be pushed on the stack before procedure calls,
  - assigned the values of procedure arguments,
  - and later restored to their original values after the procedure returns.
- Extra Pushes/Pops
- Programmers have to be very careful that each register's PUSH is matched by its appropriate POP.

```
push ebx                          ; save register values
push ecx
push esi

mov esi, OFFSET array             ; use registers as parameters
mov ecx, LENGTHOF array
mov ebx, TYPE array
call DumpMem

pop esi                           ; restore register values
pop ecx
pop ebx
```

- Stack parameters offer a flexible approach that does not require register parameters. Just before the subroutine call, the arguments are pushed on the stack.

```
push TYPE array
push LENGTHOF array
push OFFSET array
call DumpMem
```

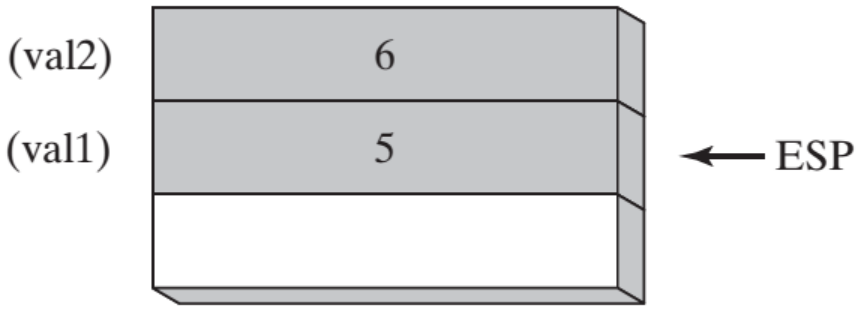
- Two general types of arguments are pushed on the stack during subroutine calls:

1. Value arguments (values of variables and constants)
2. Reference arguments (addresses of variables)

Passing by value

- When an argument is passed *by value*, a copy of the value is pushed on the stack.

```
.data
        val1 DWORD 5
        val2 DWORD 6
.code
        push val2
        push val1
        call AddTwo
```



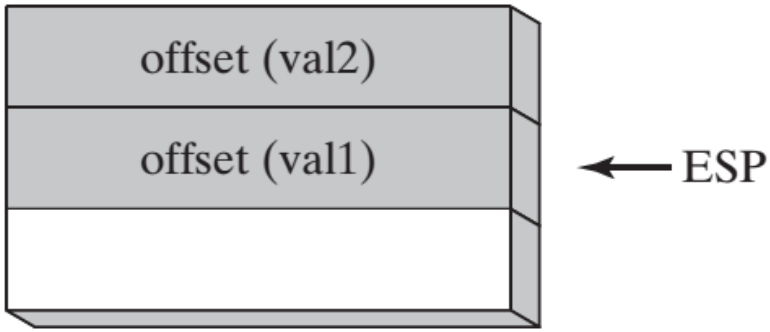
- E.g. in C++:

```
int sum = AddTwo( val1, val2 );
```

Passing by reference

- An argument passed by reference consists of the address (offset) of an object

```
push OFFSET val2
push OFFSET val1
call Swap
```



In C++:

```
Swap( &val1, &val2 );
```

Passing array

- High-level languages always pass arrays to subroutines by reference. That is, they push the address of an array on the stack. The subroutine can then get the address from the stack and use it to access the array.

```
.data
        array DWORD 50 DUP(?)
.code
        push OFFSET array
        call ArrayFill
```

Accessing Stack Parameters

- EBP is called the **base pointer** or **frame pointer** because it holds the base address of the stack frame.
- Given the following C++ function:

```
main()
{ AddTwo(5,6)}

int AddTwo( int x, int y )
{
        return x + y;
}
```

- A function call such as **`AddTwo(5,6)`** would cause the second parameter to be pushed on the stack, followed by the first parameter.

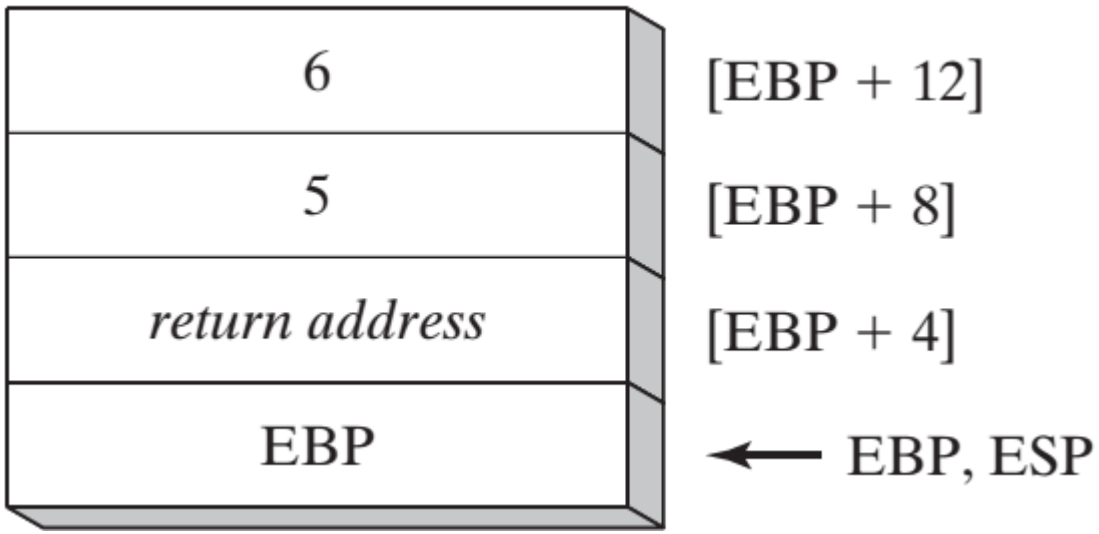
```
.code
Main PROC
PUSH 5
PUSH 6
CALL AddTwo    ; 5 first arg, 6 the second arg and ret
add
ADD ESP, 8           ; Reclaiming the space allocated
to arguments
Main ENDP

AddTwo PROC
PUSH EBP
MOV EBP, ESP                    ; Setting the central
point

MOV EAX, [EBP+8]       ;Accessing 6, the argument
ADD EAX, [EBP+12]      ; Adding 5, the second argument

POP EBP                         ; Restoring EBP
RET
AddTwo ENDP
```

- ESP would change value, but EBP would not.
- Base-Offset Addressing is used to access stack parameters.
  - EBP is the base register and the offset is a constant (***explicit stack parameters)*** .



- **The Assembly Equivalent:** In its earliest execution, **AddTwo** pushes EBP on the stack to preserve its existing value, and then EBP is set to the same value as ESP, so EBP can be the base pointer for AddTwo's stack frame:

```
AddTwo PROC
       push ebp
       mov ebp,esp
```

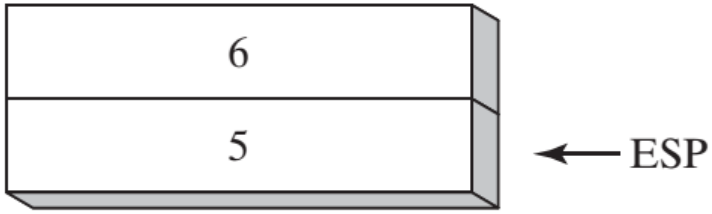
```
AddTwo PROC
       push ebp
       mov ebp,esp             ; base of stack frame
       mov eax,[ebp + 12]      ; second parameter
       add eax,[ebp + 8]       ; first parameter
       pop ebp
       ret
AddTwo ENDP
```

- When stack parameters are referenced with expressions such as `[ebp + 8]`, we call them ***explicit stack parameters***.

Cleaning Up the Stack

- There must be a way for parameters to be removed from the stack when a subroutine returns.

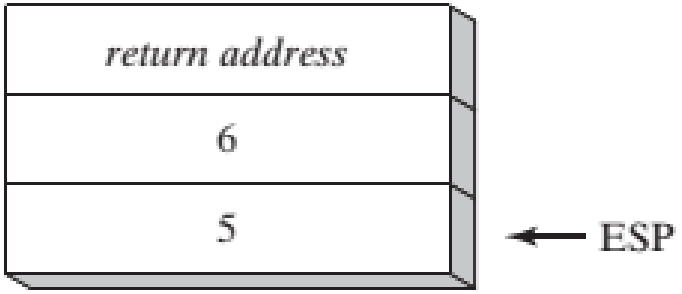
```
push 6
push 5
call AddTwo
ADD ESP, 8
```



Assuming that `AddTwo` leaves the two parameters on the stack, the following illustration shows the stack after returning from the call:

```
main PROC
 call Example1
 exit
main ENDP

Example1 PROC
  push 6
  push 5
  call AddTwo
  ret                    ; stack is corrupted!
Example1 ENDP
```



- When the RET instruction in Example1 is about to execute, ESP points to the integer 5 rather than the return address that would take it back to main

- if we were to call AddTwo from a loop, the stack could overflow. Each call uses 12 bytes of stack space:
  - 4 bytes for each parameter, plus 4 bytes for the CALL instruction's return address.

- **32-Bit Calling Conventions**

1. **The C Calling Convention:** The C calling convention is used by the C and C++ programming languages.
   - Subroutine parameters are pushed on the stack in reverse order.

- The C calling convention solves the problem of cleaning up the runtime stack in a simple way:
- When a program calls a subroutine, it follows the CALL instruction with a statement that adds a value to the stack pointer (ESP) equal to the combined sizes of the subroutine parameters.

```
Example1 PROC
        push 6
        push 5
        call AddTwo
        add esp,8       ; remove arguments from the stack
        ret
Example1 ENDP
```

2. **STDCALL Calling Convention:** Another common way to remove parameters from the stack is to use a convention named STDCALL.

- We supply an integer parameter to the RET instruction, which in turn adds integer to ESP after returning to the calling procedure.
- The integer must equal the number of bytes of stack space consumed by the procedure's parameters

```
AddTwo PROC
        push ebp
        mov ebp,esp                ; base of stack frame
        mov eax,[ebp + 12]         ; second parameter
        add eax,[ebp + 8]          ; first parameter
        pop ebp
        ret 8                      ; clean up the stack
AddTwo ENDP
```

Saving and Restoring Registers

- Subroutines often save the current contents of registers on the stack before modifying them so that the original values can be restored just before the subroutine returns.

```
MySub PROC
        push ebp                    ; save base pointer
        mov ebp,esp                 ; base of stack frame
        push ecx
        push edx                    ; save EDX
        mov eax,[ebp+8]             ; get the stack parameter
… … … .. … .. .. . .
        pop edx                     ; restore saved registers
        pop ecx
        pop ebp                     ; restore base pointer
        ret                         ; clean up the stack
MySub ENDP
```

Local variables

- Local variables are created on the runtime stack, usually below the base pointer (EBP).

```
void MySub()
{
        int X = 10;
        int Y = 20;
}
```

Variable	Bytes	Stack Offset
X	4	EBP – 4
Y	4	EBP – 8

```
MySub PROC

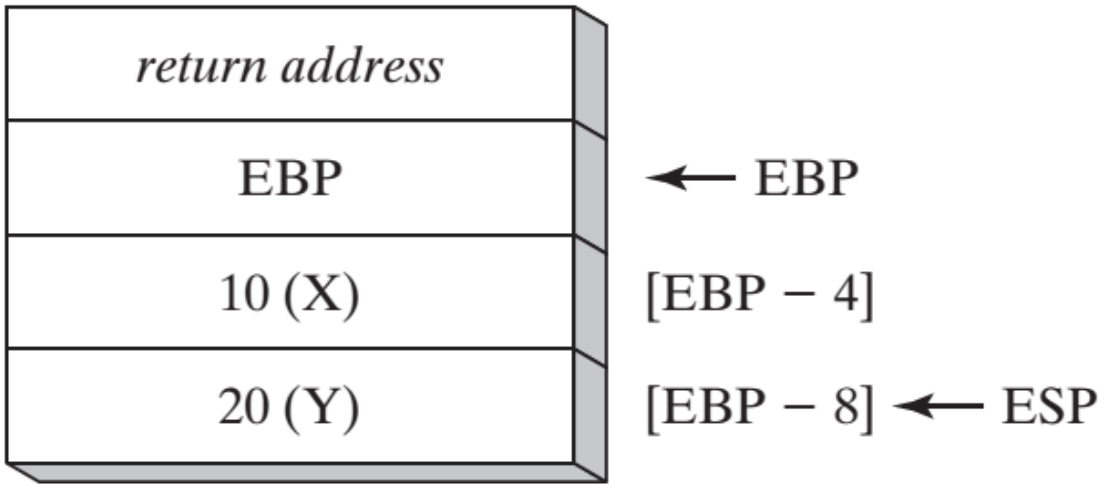
 push ebp
 mov ebp,esp
 sub esp,8                          ; create locals

 mov DWORD PTR [ebp - 4],10         ; X
 mov DWORD PTR [ebp-8],20           ; Y

 mov esp,ebp                        ; remove locals from stack

 pop ebp
 ret

MySub ENDP
```



```
Main()
{
Func1 (9)
}
Func1(int x)
{
Int y =10;
Return x+y
}
```

```
main PROC
PUSH 9
CALL Func1
ADD esp, 4
Main ENDP

Func1 PROC
PUSH ebp
Mov ebp, esp
SUB esp, 4
Mov [ebp-4] , 10
MOV eax, [ebp+8]
ADD eax, [ebp-4]
Mov esp, ebp
POP ebp
RET
Func1 ENDP
```

Enter and leave Instructions

- The **ENTER** instruction performs three operations:

1. Pushes `EBP` on the stack (`push ebp`)
2. Sets `EBP` to the base of the stack frame (`mov ebp, esp`)
3. Reserves space for local variables (`sub esp,numbytes`)

ENTER *numbytes*, *nestinglevel*

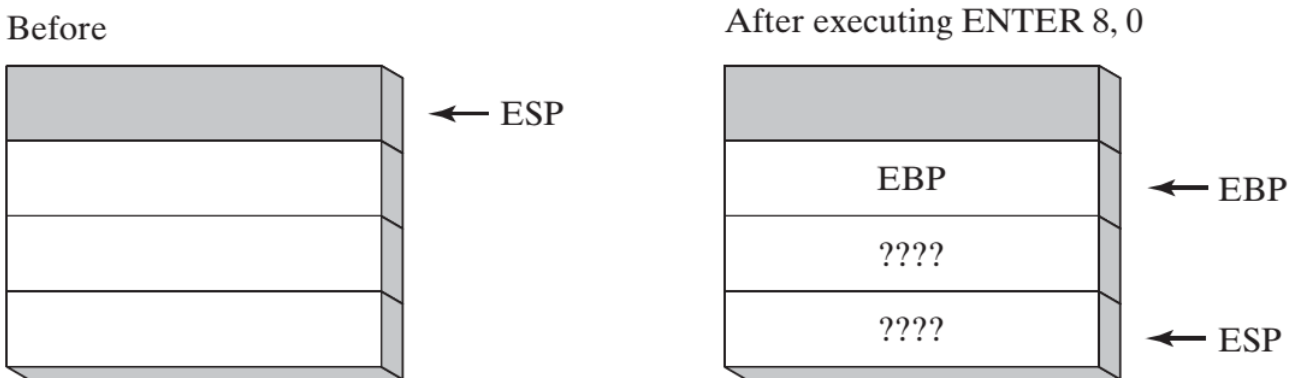
- Both the operands are immediate values,
- The first is a constant specifying the number of bytes of stack space to reserve for local variables
- The second specifies the lexical nesting level of the procedure.

- E.g. a procedure with no local variables:

```
MySub PROC
    ENTER 0,0
```

- E.g. The ENTER instruction reserves 8 bytes of stack space for local variables

```
MySub PROC
    ENTER 8,0
```



- The **LEAVE** instruction terminates the stack frame for a procedure.
  - It reverses the action of a previous ENTER instruction by restoring ESP and EBP to the values they were assigned when the procedure was called.

```
MySub PROC
        enter 8,0
        .
        .
        leave
        ret
MySub ENDP
```

LOCAL Directive

- **LOCAL** declares one or more local variables by name, assigning them size attributes.
- ENTER, on the other hand, only reserves a single unnamed block of stack space for local variables.
- If used, LOCAL must appear on the line immediately following the PROC directive.

- ```
  MySub PROC
      LOCAL var1[10]:BYTE
  ```
  ```

LEA Instruction

- The LEA instruction returns the address of an indirect operand.
- Ideally suited for use with stack parameters

```
void makeArray()
{
char myString[30];
for( int i = 0; i 30; i++
)
    myString[i] = '*';
}
```

```
makeArray PROC
 push ebp
 mov ebp,esp
 sub esp,32
 lea esi,[ebp-30]
 mov ecx,30
 L1: mov BYTE PTR [esi],'*'
 inc esi
 loop L1
 add esp,32 ; (restore ESP)
 pop ebp
 ret
makeArray ENDP
```

8.3 Recursion

- A *recursive subroutine* is one that calls itself.
- *Recursion*, the practice of calling recursive subroutines, can be a powerful tool when working with data structures that have repeating patterns.
- Useful recursive subroutines always contain a terminating condition.
- When the terminating condition becomes true, the stack unwinds when the program executes all pending RET instructions.

```
INCLUDE Irvine32.inc

.data
       endlessStr BYTE "This recursion never stops",0

.code
  main PROC
    call Endless
    exit
  main ENDP

  Endless PROC
    mov edx,OFFSET endlessStr
    call WriteString
    call Endless
    ret                                ; never executes
  Endless ENDP

END main
```

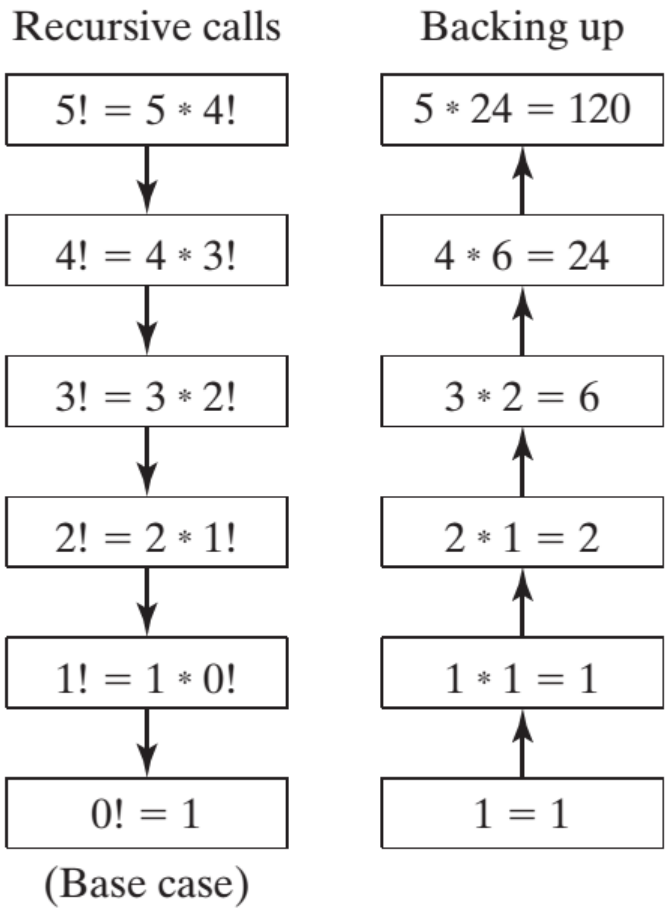
```
INCLUDE Irvine32.inc
.code
main PROC
        mov ecx,5                       ; count = 5
        mov eax,0                       ; holds the sum
        call CalcSum                    ; calculate sum
        L1: call WriteDec               ; display EAX
        call Crlf                       ; new line
        exit
main ENDP

CalcSum PROC
        cmp ecx,0                       ; check counter value
        jz L2                           ; quit if zero
        add eax,ecx                     ; otherwise, add to sum
        dec ecx                         ; decrement counter
        call CalcSum                    ; recursive call
        L2: ret
CalcSum ENDP
end Main
```

Pushed on Stack	Value in ECX	Value in EAX
L1	5	0
L2	4	5
L2	3	9
L2	2	12
L2	1	14
L2	0	15

Calculating a Factorial

```
int function factorial(int n)
{
        if(n == 0)
           return 1;
        else
           return n * factorial(n-1);
}
```

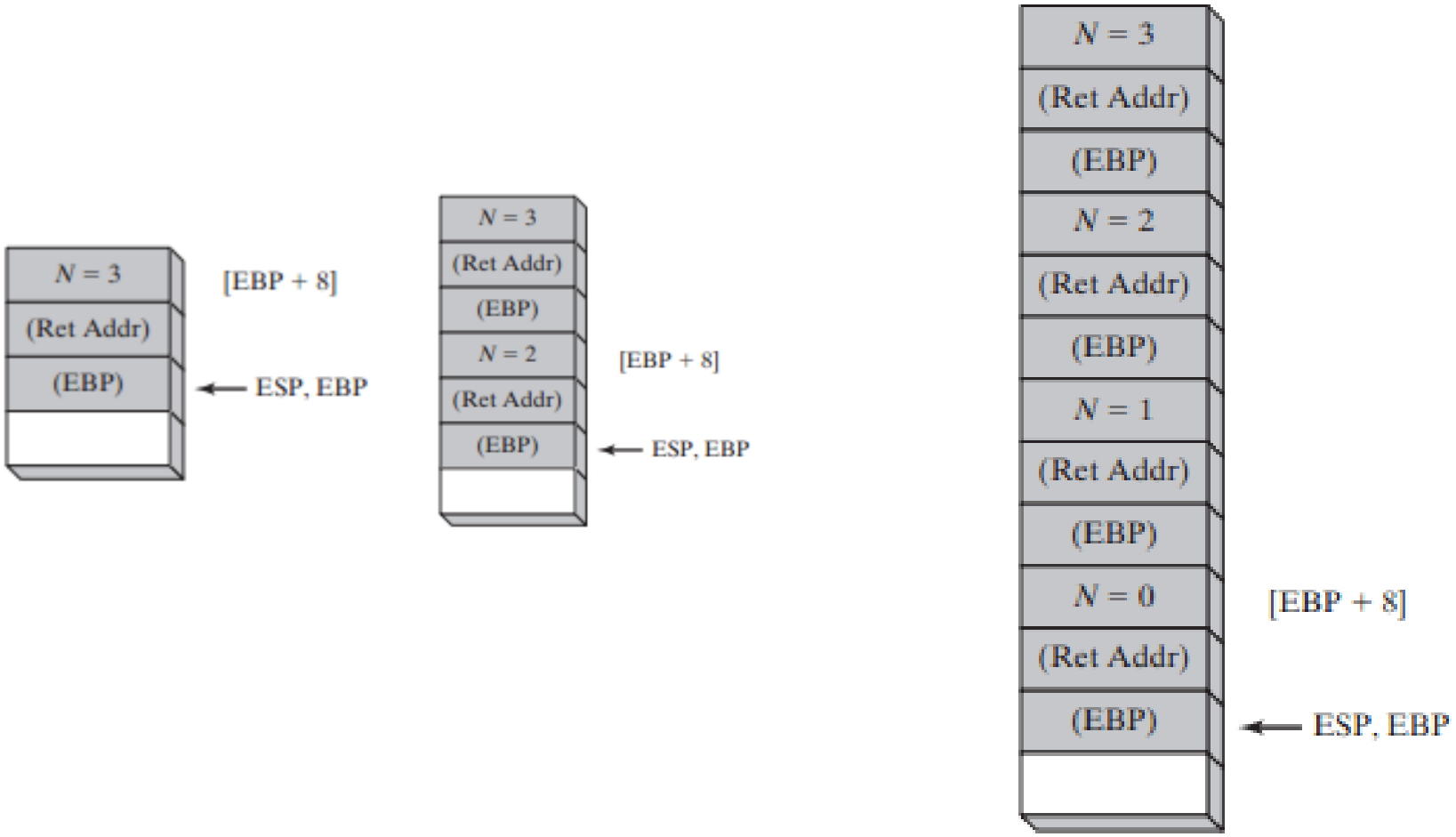


```
; Calculating a Factorial (Fact.asm)
INCLUDE Irvine32.inc
.code
main PROC
push 5 ; calc 5!
call Factorial ; calculate factorial (EAX)
call WriteDec ; display it
call Crlf
exit
main ENDP
;-----------------------------------------------------
Factorial PROC
; Calculates a factorial.
; Receives: [ebp+8] = n, the number to calculate
; Returns: eax = the factorial of n
;-----------------------------------------------------
push ebp
mov ebp,esp
mov eax,[ebp+8] ; get n
cmp eax,0       ; n > 0?
ja L1           ; yes: continue
mov eax,1       ; no: return 1 as the value of 0!
jmp L2          ; and return to the caller
```

```
L1: dec eax
push eax        ; Factorial(n  1)
call Factorial

; Instructions from this point on execute when each
; recursive call returns.

ReturnFact:
mov ebx,[ebp+8]     ; get n
mul ebx             ; EDX:EAX = EAX * EBX
L2: pop ebp         ; return EAX
ret 4               ; clean up stack
Factorial ENDP
END main
```


N = 3
(Ret Addr)
(EBP)
[EBP + 8]
ESP, EBP
N = 3
(Ret Addr)
(EBP)
N = 2
(Ret Addr)
(EBP)
[EBP + 8]
ESP, EBP
N = 3
(Ret Addr)
(EBP)
N = 2
(Ret Addr)
(EBP)
N = 1
(Ret Addr)
(EBP)
N = 0
(Ret Addr)
(EBP)
[EBP + 8]
ESP, EBP

```
int function factorial(int n)
{
        if(n == 0)
           return 1;
        else
           return n * factorial(n-1);
}
```

```
Main PROC
  ENTER 0,0
  Mov ECX, 5
  MOV eax, 5; number to calculate factorial
  INVOKE factorial, eax

 Add Esp, 4
 LEAVE
 ret
Main ENDP
```

```
factorial PROC, N: DWORD
  push ebp
  mov ebp,esp
  DEC ECX
  CMP ECX, 0
  JZ L1
  DEC N
  MUL N ;
  INVOKE factorial, N
  L1: POP EBP
  Add esp, 4
  ret
Main ENDP
```

Stack Address	Stack Content	;Comments	Stack Frame
0000 FC12	Ret Address	To the system	Main
0000 FC0E	0000 0000	<-ebp	Main
0000 FC0A	5	Arg from main	1st (Factorial)
0000 FC06	Ret Address	To main	1st (Factorial)
0000 FC02	0000 FC0E	<-ebp(previous)	1st (Factorial)
0000 FBFE	4	Arg from fact:	2nd (Factorial)
0000 FBFA	RET Address	To fact_1	2nd (Factorial)
0000 FBF6	FC02	<-ebp(previous)	2nd (Factorial)
0000 FBF2	3	Arg from fact:	3rd (Factorial)
0000 FBEE	RET Address	To fact_2	3rd (Factorial)
0000 FBEA	FBF6	<-ebp(previous)	3rd (Factorial)
0000 FBE6	2	Arg from fact:	4th (Factorial)
0000 FBE2	RET Address	To Fact_3	4th (Factorial)
0000 FBDE	FBEA	<-ebp(previous)	4th (Factorial)
0000 FBDA	1	Arg from fact:	5th (Factorial)
…	…		…

Assuming EBP = 0000 0000 Initially, and stack starts at 0000 FC12h